

Learning goals: `round()` and `print()` functions, Sequences, Missing values, Vectors, Matrices, Vector functions

EXERCISES

- (1) **An R quirk, or is it?** Let's try to understand the function `round()` and `print()` using the following examples and see why the output of `round(123456789,digits=3)` did not give the expected output.
 - (i) `round(12.1343,digits=3)`
 - (ii) `round(123.12344,digits=3)`
 - (iii) `round(1234.12344,digits=3)`
 - (iv) `round(12345.12344,digits=3)` What happened here? The output here is `12345.12`! R has limits on the number of digits it displays for large numbers. There are two ways to deal with this.
 - (v) One way is to change the environment options of number of digits displayed: `options(digits=15)` and then try printing the above again.
 - (vi) Second way is to use `formatC` function.
`formatC(round(12345.12344,digits=3),format="f",digits=3)`
 - (vii) What is the default format for `print` function? Try `print(1234.12344)`?
 - (viii) What about `print(1234.723,digits=3)`? The `digits` option here refers to the number of significant digits. If the number of digits is less than or equal to the number of digits before the decimal point, then `print()` retains the nearest rounded integer (here it will be rounded to 1235). If the `digits` value is greater than the number of digits before the decimal point, then the appropriate number of digits after the decimal point are retained. Try `print(1234.723,digits=5)`
 - (ix) The same rule applies to `round` function. Try `round(123456788.123,digits=3)`
 - (x) What about `print(round(123456788.123,digits=2),digits=20)`?
 - (xi) Also try `print(round(123456789.1234,digits=4),digits=20)` to understand the `round()` function better.
 - (xii) `paste("Hello World")` and `paste("Hello","World")`
 - (xiii) `paste(1:10)` and `paste(1:10)[4]`
 - (xiv) convert strings back to numbers as `numeric(paste(1:10))`
 - (xv) `paste(1:10,collapse=".")`
 - (xvi) Pasting objects of different lengths `paste(c("Hello","World"),1:10,sep="-")`
, `print(paste("Hello",1:10,sep="-"))`
- (2) **Generating sequences**
 - (i) `0:10`
 - (ii) `15:5`
 - (iii) `seq(0,1.5,0.1)`
 - (iv) `seq(6,4,-0.2)`
 - (v) `N <- c(55,76,92,103,84,88,121,91,65,77,99)`
 - (vi) Let's say you want to plot `N` against a sequence that starts at 0.04 in steps of 0.01. `seq(from=0.04,by=0.01,length=11)`. A more elegant way is to use the `along` argument and specify the vector `N` whose length has to be matched.
`seq(0.04,by=0.01,along=N)`
 - (vii) Also try `seq(from=0.04,to=0.14,along=N)` Does this match with the above sequence?
 - (viii) The `sequence` function (as opposed to `seq`) is useful to make a vector of sequences of unequal lengths. `sequence(c(4,3,4,4,4,5))`

- (ix) Generating repeats: `rep(9,5)`, `rep(1:4,2)`, `rep(1:4,each=2)`, `rep(1:4,each=2,times=3)`, `rep(1:4,1:4)` (The first argument of `rep()` is the vector, second argument is times by default and the third argument is 'each'),
 - (x) `rep(1:4,c(4,1,4,2))`, `rep(c("cat","dog","goldfish","rat"), c(2,3,2,1,3))`
 - (xi) `seq(-1,1,by=0.1)` or `seq(-1,1,0.1)`
 - (xii) `seq(-1,1,length=7)`
 - (xiii) Generate a set of numbers from -1 to 1 with an interval of 0.1 without using `seq()`
- (3) **Missing values, infinity and NaN, NA**
- (i) `3/0`
 - (ii) `exp(-Inf)`
 - (iii) `(0:3)**Inf`
 - (iv) `0/0`
 - (v) `Inf - Inf`
 - (vi) `Inf/Inf`
 - (vii) `is.finite(10)`
 - (viii) `is.infinite(10)`
 - (ix) `is.infinite(Inf)`
 - (x) `y<- c(4,NA,7)` followed by `y=="NA"` and `is.na(y)`
 - (xi) Strip y of the NA entry: `y[!is.na(y)]`
 - (xii) Carry out the following exercise:
`c1<- c(1,2,3,NA)`
`c2<- c(5,6,NA,8)`
`c3<- c(9,NA,11,12)`
`c4<- c(NA,14,15,16)`
`full.frame <- data.frame(c1,c2,c3,c4)`
`reduced.frame <- full.frame[! is.na(full.frame$c1),]`
 This will remove the row containing NA with respect to column 1. Functions such as `mean(x)` will not work with missing values. So use `mean(x,na.rm=T)`
 - (xiii) Get the indices of a vector where NA is present:
`v <- c(1:6,NA,NA,9:12)`
`seq(along=v)[is.na(v)]`
`which(is.na(v))`
- (4) **Vectors**
- (i) Vector assignment and queries: `vec <- c(4,7,6,5,6,7)` followed by `class(vec)`, `length(vec)`, `min(vec)`, `max(vec)`
 - (ii) Vector creation using keyboard input: `vec <- scan()` followed by series of entries and an extra **Enter** so as to tell R that the vector is now complete. In most situations vectors are created by reading in data from a file.
 - (iii) Vector subscripts: `vec[4]` (Indexing starts from 1 unlike in Python where it is 0)
 - (iv) Extracting multiple elements using two ways: `ind <- c(2,3,6)` followed by `vec[ind]` OR `vec[c(2,3,6)]`
 - (v) Drop elements using minus symbol: `vec[-1]` will remove the 1st element from `vec` (check this!)
 - (vi) What does `vec[-length(vec)]` do?
 - (vii) Write a function called `trim` to remove the largest two and smallest two values from a vector called `x`. Use this function on the above `vec` vector and print the result.
 - (viii) Use sequences to extract elements: `vec[1:3]`, `vec[seq(2,length(vec),2)]` (What is a different way to achieve this?)

- (ix) Working with logical subscripts: Create a vector `x <- 0:10`. We are interested in finding the sum of values of `x` that are less than 5. How do we do this? (There are at least 2 ways of doing this.)
- (x) Along similar lines, work out the logic to find out the sum of three largest values in a vector.
- (xi) Finding index of vector corresponding to maxm or minm value: `which.max(x)`
OR `which.min(x)`
- (xii) Combining vectors as columns or rows: `cbind(1:10,10:1)`, `rbind(1:10,10:1)`
- (xiii) Basic operations with vectors:

```
X <- c(1:10)
X
Y <- c(1:10*5)
Y
X*Y
X+Y
X/Y
X^Y
log(X)
exp(Y)
```

- (5) **Matrices/Dataframes/Arrays** are multi-dimensional objects. Let's build a matrix with dimensions $2 \times 4 \times 3$. Printing `y` outputs three 2-dimensional tables. What does `y` look like when you change the dimensions to $3 \times 2 \times 4$?

```
y <- 1:24
dim(y) <- c(2,4,3)
y
```

Matrices are 2D arrays containing numbers, whereas dataframes are 2D lists containing potentially a mix of numbers, text or logical variables in different columns. First subscript refers to row, and second subscript refers to the column index. Let's build a matrix using the `matrix` function – this essentially converts vectors to matrices.

- (i) `X <- matrix (c(1,0,0,0,1,0,0,0,1),nrow=3)` followed by `print(X)` Note how the column indices have `[,1]` `[,2]` etc, the use of a blank subscript to denote "all of". `[,1]` means "all of" row 1.
 - (ii) Another way to convert a vector to matrix is to provide the vector object with two dimensions (rows and columns) using the `dim` function:

```
vector <- c(1,2,3,4,4,3,2,1)
V <- matrix(vector,byrow=T,nrow=2) #method 1
dim(vector) <- c(4,2) #method 2
is.matrix(vector)
```
- (6) **Vector functions** This is one of the biggest strengths of R because one can evaluate functions over entire vectors without the need to invoke loops and subscripts. So learn this well!
- (i) Common functions used: `min(vec)`, `max(vec)`, `sum(x)`, `range(x)`, `sort(x)`
 - (ii) The function `colMeans(M)` evaluate the mean of each column of dataframe/matrix `M`.
 - (iii) Matrix multiplication has applications throughout biology, ranging from gene expression and molecular biology to community ecology and image analysis.

```
Z <- X[1:4] \%o\% Y[1:3]
Z
YoX <- Y[1:3] \%o\%X[1:4]
```

```

YoX
t(Z) #transpose
t(YoX)
X \%*\% Y #dot product
sum(X*Y) #another way to carry out dot product
crossprod(X[1:4],Z)
diag(4) #identity matrix
(iv) class(X)

```

- (7) **Lists/dataframes: Introduction** Lists are useful for mixing data types, and can combine different dimensions of cells, vectors, and higher order arrays. And R uses list objects to store output. So knowing lists is useful to interpret output of R functions.

```

Xvec <- c("Jayashree","Nagesh",119,"quantum",2000,2003,2006,2012,2017,2023)
Xvec
Mylist <- list(name="JN",mixedlist=Xvec,numbers=1:10)
Mylist
Mylist$numbers #Use $ to subset by name
Mylist[3]
Mylist[[3]] #how is this different from Mylist[3]?

```